

Aula 21: Introdução ao Machine Learning Clássico

Algoritmos do Scikit-Learn, Hiperparâmetros e Isolamento de Dados

1 O Paradigma do Aprendizado de Máquina

Diferente da programação tradicional, onde regras rígidas são codificadas manualmente para processar dados de entrada, no Aprendizado de Máquina (*Machine Learning - ML*) o computador aprende os padrões diretamente a partir dos dados.

- **Aprendizado Supervisionado:** O algoritmo aprende uma função de mapeamento de entradas para saídas rotuladas. Divide-se em:
 - **Classificação:** A variável alvo é discreta/categórica (ex: predição de inadimplência: pago ou não pago).
 - **Regressão:** A variável alvo é numérica contínua (ex: predição do salário de um profissional com base em anos de experiência).
- **Aprendizado Não Supervisionado:** O algoritmo detecta padrões ocultos sem a presença de rótulos predefinidos (ex: agrupamento de clientes por perfil de compra).

2 Algoritmos Clássicos no Scikit-Learn

Para resolver problemas práticos de classificação e regressão, o ecossistema Python disponibiliza classes consolidadas dentro da biblioteca `scikit-learn`. Antes de construir qualquer modelo, a etapa fundamental é isolar uma parte dos dados para validar se o algoritmo realmente aprendeu os padrões ou apenas os memorizou.

2.1 Preparação Básica: Divisão de Treino e Teste

```
1 from sklearn.model_selection import train_test_split
2
3 # Separando 80% dos dados para treino e 20% para teste
4 X_train, X_test, y_train, y_test = train_test_split(
5     X, y, test_size=0.20, random_state=42
6 )
```

2.2 Regressão Logística (LogisticRegression)

Apesar do nome "regressão", é um algoritmo linear estatístico amplamente empregado na **classificação binária**. Ele calcula a probabilidade matemática de uma amostra pertencer

à classe positiva mapeando a combinação linear das variáveis em uma curva sigmoide (função logística):

$$P(y = 1|z) = \frac{1}{1 + e^{-z}} \quad \text{onde} \quad z = \beta_0 + \beta_1 x_1 + \dots + \beta_n x_n \quad (1)$$

Para converter essa probabilidade contínua (ex: 0.72) em uma classe discreta, o algoritmo utiliza um **limiar de decisão (Threshold)**, que por padrão é 0.5. Se $P(y = 1) \geq 0.5$, a classe predita é 1; caso contrário, é 0. As classificações podem ser posteriormente avaliadas utilizando métricas como Acurácia, Precisão, Recall e F1-Score através da Matriz de Confusão.

Justificativa de Escolha: É um modelo extremamente interpretável, ideal para cenários regulamentados (análise de crédito, diagnósticos médicos). Ele permite compreender o impacto direto de cada feature por meio de seus coeficientes (*odds ratio*).

Analogia do Mundo Real: A Balança de Julgamento

Imagine um juiz pesando evidências em uma balança. Cada variável do modelo é um "peso" (alguns pesam a favor da condenação, outros a favor da inocência). A função logística converte esses pesos em uma probabilidade de 0 a 100%. O *threshold* (limiar) de 0.5 é o momento exato em que a balança pende de um lado para o outro de forma definitiva, forçando a sentença final ("Culpado" ou "Inocente").

```
1 from sklearn.linear_model import LogisticRegression
2
3 # Instancia o modelo definindo um threshold de regularizacao (C)
4 modelo_log = LogisticRegression(C=1.0, max_iter=1000)
5
6 # Treinamento do modelo
7 modelo_log.fit(X_train, y_train)
8
9 # Coeficientes aprendidos (impacto de cada feature)
10 print(modelo_log.coef_)
```

A curva logística resultante desse mapeamento é apresentada graficamente na Figura 1. Note que as escalas foram ajustadas de forma proporcional para representar o comportamento suave da função sigmoide sem distorção visual.

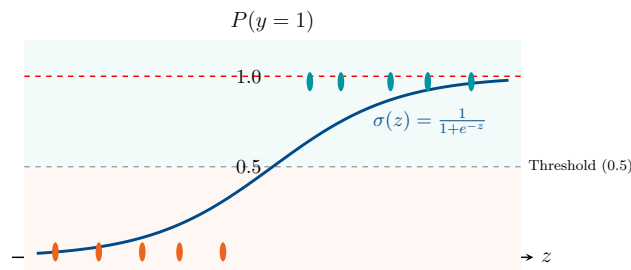


Figura 1: Curva Logística (Sigmoide) com Proporções de Escala Ajustadas

Exemplo Prático: Análise de Risco de Crédito (Credit Scoring)

Um banco precisa prever se um cliente vai honrar o empréstimo (1) ou dar calote (0). A Regressão Logística processa o histórico do cliente e devolve uma probabilidade. O

resultado típico da avaliação desse modelo é demonstrado em uma Matriz de Confusão, cruzando o que o modelo previu contra o que realmente aconteceu no mundo real.

		Previsto: Pagou (1)		Previsto: Calote (0)	
Real: Pagou (1)	Verdadeiro	Positivo (Pagou e Previo Pagou)		Falso Positivo (Calote, previu Pagou)	
	Falso	Falso Negativo (Pagou, previu Calote)	15	Verdadeiro Negativo (Calote e Previo Calote)	95
Real: Calote (0)					

Figura 2: Matriz de Confusão para a Detecção de Inadimplentes

2.3 Árvores de Decisão (DecisionTreeClassifier / Regressor)

Modelos não-paramétricos que realizam partições recursivas criando regras lógicas "se-então". Para selecionar a melhor divisão (*split*), o algoritmo calcula métricas de impureza como o **Índice de Gini** ou a **Entropia**. O algoritmo testa todos os cortes possíveis e escolhe aquele que resulta em "folhas" mais puras (onde as amostras pertencem majoritariamente a uma única classe).

Justificativa de Escolha: Dispensa premissas matemáticas complexas (não exige linearidade, normalidade ou escalonamento das features) e seu modelo de decisão visualizável facilita auditorias de governança. O grande risco, entretanto, é o *overfitting* se crescer demasiadamente.

Analogia do Mundo Real: O Jogo "Cara a Cara"

Funciona exatamente como o clássico jogo de tabuleiro. A melhor estratégia para vencer não é fazer uma pergunta específica demais ("Seu personagem usa óculos vermelhos?"), mas sim a pergunta que divide o tabuleiro pela metade ("Seu personagem é homem?"). A Árvore de Decisão calcula o Gini para descobrir matematicamente qual "pergunta" elimina a maior quantidade de incerteza (ganho de informação) a cada rodada.

```

1 from sklearn.tree import DecisionTreeClassifier, plot_tree
2
3 # Arvore limitada para evitar overfitting
4 arvore = DecisionTreeClassifier(max_depth=4, criterion='gini')
5 arvore.fit(X_train, y_train)
6
7 # Funcao nativa para visualizar as regras logicas
8 plot_tree(arvore, filled=True, feature_names=X.columns)

```

A Figura 3 simula o *output* nativo do modelo treinado pelo Scikit-Learn, contendo os metadados matemáticos reais presentes nos nós de aprendizado.

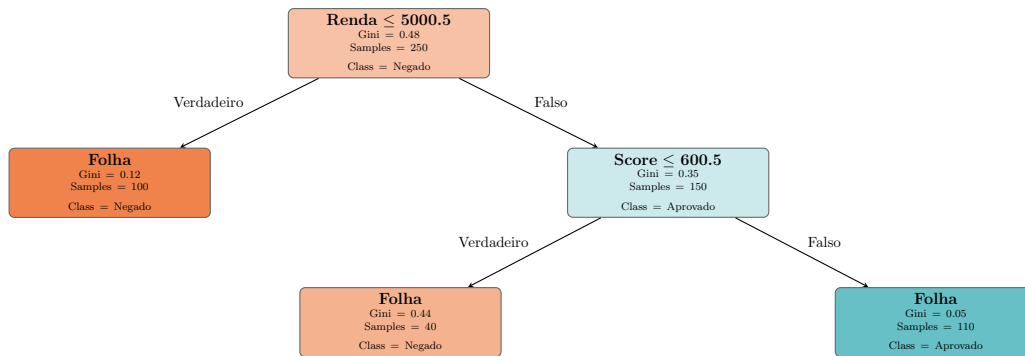


Figura 3: Visualização de uma Árvore de Decisão com Índice de Gini e Partições

Exemplo Prático: Triage Médica de Pronto-Socorro

Durante um plantão caótico, enfermeiros precisam determinar a gravidade do paciente em segundos. A Árvore de Decisão permite que o hospital gere um protocolo de atendimento instantâneo extraído de anos de dados de pacientes.

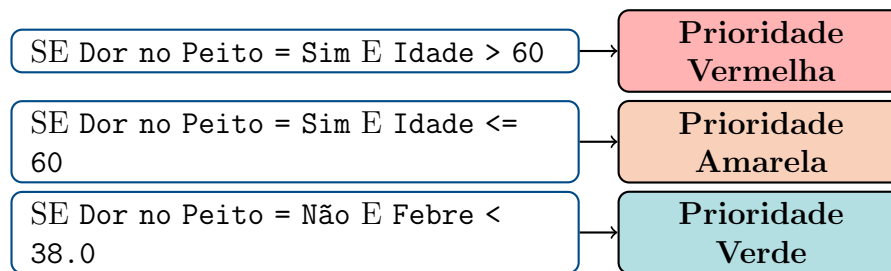


Figura 4: Tabela Visual de Regras Extraídas de uma Árvore de Decisão Médica

2.4 Random Forests (RandomForestClassifier / Regressor)

Um poderoso algoritmo baseado em conjunto (*Ensemble Method*) que constrói uma "floresta" de múltiplas árvores de decisão paralelas. Utiliza a técnica de *Bagging* (Bootstrap Aggregating) para treinar cada árvore em uma amostragem aleatória dos dados (com reposição), submetendo também subconjuntos aleatórios de colunas (features) para cada árvore.

A previsão final é consolidada pela média (regressão) ou pela **votação majoritária** (classificação). O princípio baseia-se na sabedoria da multidão: muitos modelos simples errarão de formas diferentes, e a fusão de suas respostas convergirá para a decisão correta.

Trade-off Viés-Variância: Enquanto uma única Árvore de Decisão possui alto risco de memorizar os dados (alta variância), a Random Forest neutraliza essa vulnerabilidade. Ela reduz drasticamente a variância estatística do modelo mantendo o viés controlado, oferecendo robustez fenomenal contra *outliers* e instabilidades.

Analogia do Mundo Real: O Conselho Médico

Se você consultar apenas um médico sobre um diagnóstico complexo, ele pode errar por viés de sua especialidade (*Overfitting* da árvore isolada). Na Random Forest, formamos um conselho com 100 médicos. Entregamos partes diferentes e incompletas dos exames para cada um deles (*Bagging*). Ao final, mesmo que alguns errem feio, a decisão tomada por votação democrática da maioria esmagadora diluirá os erros individuais, produzindo um laudo final extremamente assertivo.

```
1 from sklearn.ensemble import RandomForestClassifier
2
3 # Criando uma floresta com 100 arvores paralelas
4 floresta = RandomForestClassifier(n_estimators=100, random_state=42)
5 floresta.fit(X_train, y_train)
6
7 # Identificando as features mais importantes para o negocio
8 importancias = floresta.feature_importances_
```

A Figura 5 ilustra o fluxo mecânico da floresta aleatória e sua agregação final de respostas.

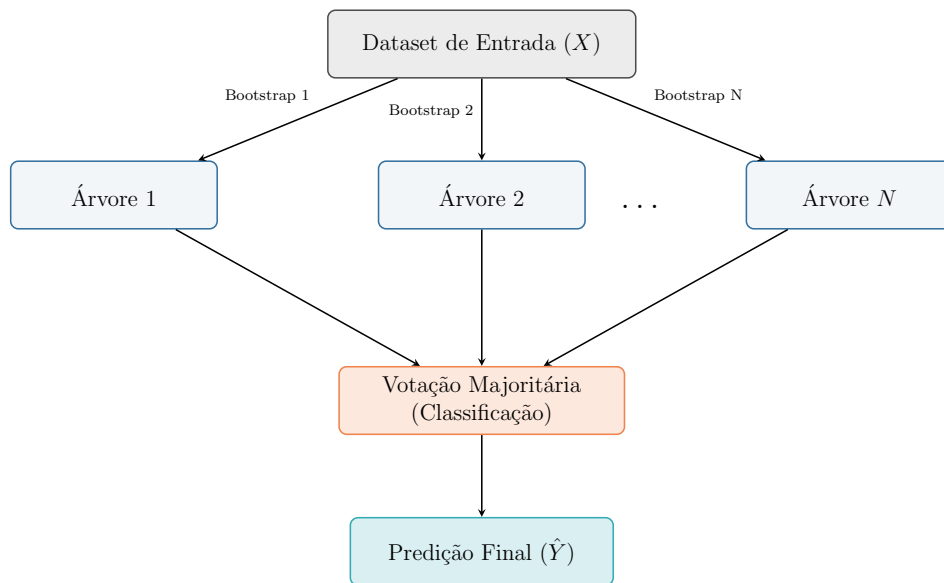


Figura 5: Arquitetura e Fusão de Decisões de uma Random Forest

Exemplo Prático: Previsão de Cancelamento de Assinatura (Churn) em Telecom

Empresas de internet perdem milhões anualmente quando clientes insatisfeitos cancelam seus planos. Utilizando centenas de variáveis de CRM, a Random Forest consegue isolar com maestria quais fatores comportamentais têm maior impacto direto na quebra de contrato.

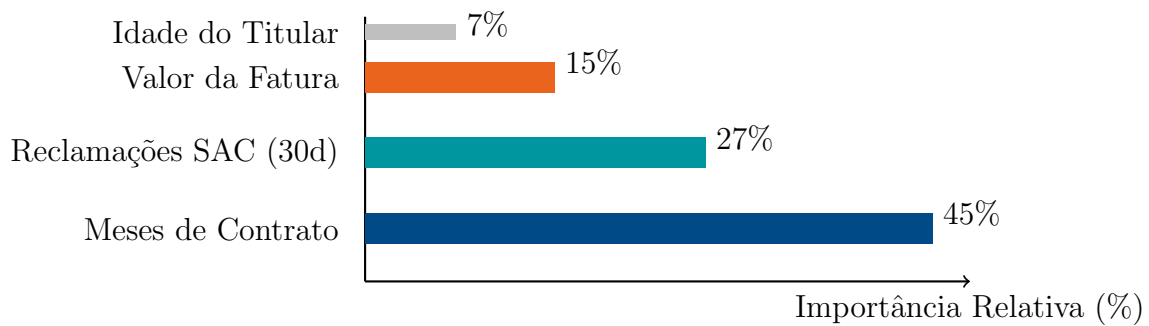


Figura 6: Gráfico de Feature Importance Extraído da Random Forest

3 Seleção de Parâmetros (Hiperparâmetros)

Nas bibliotecas de IA, diferenciamos **parâmetros** (valores ajustados internamente pelo algoritmo durante o treino, como coeficientes da regressão) dos **hiperparâmetros** (parâmetros estruturais definidos pelo cientista de dados antes do treinamento do modelo).

3.1 Hiperparâmetros Críticos do Scikit-Learn

- **Em Regressão Logística:** O parâmetro `C` controla a força de regularização de forma inversa (valores menores de `C` aplicam penalizações maiores de L1/L2 para evitar overfitting).
- **Nas Árvores de Decisão:** `max_depth` (profundidade máxima permitida para a árvore) e `min_samples_split` (mínimo de amostras necessárias para dividir um nó interno) controlam a complexidade do modelo.
- **No Random Forest:** Além dos parâmetros de árvore, `n_estimators` define o número de árvores a serem criadas no ecossistema da floresta.

3.2 Busca Sistemática e Validação Cruzada

Ajustar o modelo diretamente na base de teste resultaria em vazamento de hiperparâmetros (viés de ajuste de teste). Para contornar isso, os alunos devem usar classes como `GridSearchCV` e aplicar a **Validação Cruzada** (*K-Fold Cross-Validation*) estritamente sobre os dados de treino.

No K-Fold, o algoritmo divide a base de treinamento em K pedaços. Ele repete o treino K vezes, sempre isolando um pedaço diferente para validação, assegurando que o modelo prove seu valor em múltiplos cenários inéditos antes de adotar um parâmetro.

Analogia do Mundo Real: A Preparação para o ENEM

Se um aluno estudar pelas exatas mesmas provas antigas que ele vai fazer amanhã, ele tirará nota 10 (decorou o gabarito = *Overfitting*). O K-Fold é como separar o banco de exercícios em 5 cadernos. O aluno estuda 4 cadernos e resolve o 5º como simulado. Esse processo repete 5 vezes, revezando o caderno do simulado. Se ele for bem na média de todos os simulados, prova-se o verdadeiro aprendizado em vez da memorização!

```

1 from sklearn.model_selection import GridSearchCV
2
3 parametros = {
4     'max_depth': [3, 5, 10],
5     'min_samples_split': [2, 5, 10]
6 }
7
8 # Realiza o 5-Fold Cross Validation
9 busca = GridSearchCV(DecisionTreeClassifier(), parametros, cv=5)
10 busca.fit(X_train, y_train)
11
12 print(f"Melhor parametro: {busca.best_params_}")

```

Dados de Treinamento (Particionados para Busca de Hiperparâmetros)

Iteração 1:	Teste (Val)	Treino	Treino	Treino	Treino
Iteração 2:	Treino	Teste (Val)	Treino	Treino	Treino
Iteração 3:	Treino	Treino	Teste (Val)	Treino	Treino
Iteração 4:	Treino	Treino	Treino	Teste (Val)	Treino
Iteração 5:	Treino	Treino	Treino	Treino	Teste (Val)

Figura 7: Ilustração do Processo de 5-Fold Cross-Validation

4 O Perigo do Data Leakage (Vazamento de Dados)

O vazamento de dados ocorre quando informações do conjunto de teste ou do futuro entram inadvertidamente no conjunto de treino durante a etapa de limpeza e preparação.

Se calcularmos estatísticas globais (como média de imputação do `SimpleImputer` ou limites de escala do `StandardScaler`) sobre todo o conjunto de dados **antes** de realizar a partição de treino e teste, o modelo estudará com dados contaminados e apresentará uma performance inflada artificialmente.

Para evitar isso, dividimos os dados em Treino (80%) e Teste (20%) antes de qualquer transformação. O `Pipeline` do Scikit-learn automatiza esse isolamento, permitindo que o `.fit()` calcule os parâmetros exclusivamente sobre o treino e os aplique de forma blindada no teste via `.transform()`.

Analogia do Mundo Real: A Prova com Vazamento de Gabarito

O *Data Leakage* ocorre quando o algoritmo "ouve" discussões de corredor sobre as respostas da prova antes dela acontecer (aplicar a média global de imputação em *todos* os dados juntos antes da divisão). O modelo vai tirar notas altíssimas no seu laboratório, mas fracassará no mundo real. O `Pipeline` atua trancando o conjunto de Treino e o de Teste em salas completamente separadas, garantindo que o algoritmo só seja testado sob uma rigorosa prova cega.

```

1 from sklearn.pipeline import make_pipeline

```

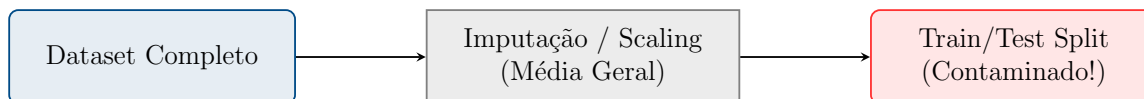
```

2 from sklearn.preprocessing import StandardScaler
3 from sklearn.impute import SimpleImputer
4
5 # Encadeia as etapas na ordem exata de execucao
6 pipeline_seguro = make_pipeline(
7     SimpleImputer(strategy='median'),
8     StandardScaler(),
9     LogisticRegression()
10 )
11
12 # Isola as transformacoes apenas para o X_train
13 pipeline_seguro.fit(X_train, y_train)
14
15 # Avaliacao segura (chama o transform implicitamente)
16 score = pipeline_seguro.score(X_test, y_test)

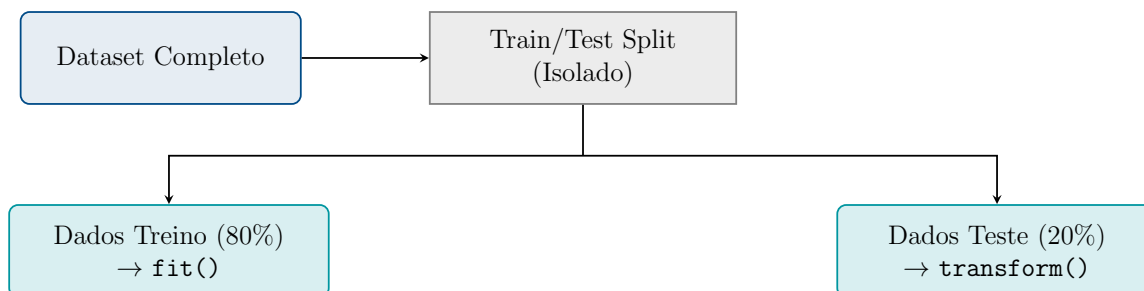
```


5 Visualização do Fluxo de Preparação de Dados

5.1 Fluxo Incorreto (Gera Data Leakage)



5.2 Fluxo Correto (Blindagem por Pipeline)



6 Atividade Prática em Sala

Utilizando as pastas do workspace, navegue até a pasta Aula 13 e execute o Jupyter Notebook `lab_13_pipelines_intro.ipynb`.

- Avalie a diferença de performance de classificação aplicando modelos diferentes (`LogisticRegression` vs `RandomForestClassifier`).
- Ajuste hiperparâmetros como a profundidade máxima das árvores no pipeline e documente a curva de aprendizado do modelo.